

Riassunto Basi di dati

Angelo Vaccaro

1 Tecnologia dei DBMS e Gestione Transazionale

I sistemi per la gestione di basi di dati (DBMS) nascono dalla necessità di gestire grandi quantità di dati in modo efficiente, sicuro e affidabile. Sotto il cofano di un moderno DBMS non troviamo una semplice collezione di file d'archivio, bensì un'architettura software stratificata che deve mediare costantemente tra due mondi hardware opposti: la memoria centrale (RAM), caratterizzata da altissima velocità ma volatile, e la memoria di massa (il disco), persistente ma intrinsecamente lenta. Il compito principale del modulo transazionale è far sì che le applicazioni possano operare sui dati a un alto livello di astrazione, senza doversi preoccupare dei potenziali conflitti tra utenti concorrenti o di improvvisi guasti hardware che rischierebbero di corrompere le informazioni.

1.1 Il Concetto di Transazione e il suo Ciclo di Vita

Nel modello relazionale, l'interazione con i dati non avviene quasi mai tramite operazioni isolate. Un'applicazione esegue solitamente sequenze di letture e modifiche che possiedono un significato logico unitario, come ad esempio il trasferimento di denaro tra due conti correnti o la prenotazione di un posto su un volo. Questa sequenza di azioni primitive di lettura (*read*) e scrittura (*write*) prende il nome di transazione.

Dal punto di vista della sintassi SQL, una transazione viene formalmente aperta tramite un comando di inizio (come **BEGIN TRANSACTION**) e si chiude dichiarando il consolidamento delle modifiche con il comando **COMMIT**, oppure richiedendo l'annullamento totale delle stesse tramite il comando **ROLLBACK** (o **ABORT**). Durante la sua esecuzione, la transazione attraversa diversi stati logici:

- **Attivo (Active):** È lo stato iniziale in cui la transazione si trova mentre il motore SQL esegue i vari comandi inviati dall'applicazione. I dati vengono letti dal disco o scritti temporaneamente in memoria.
- **Parzialmente Impegnato (Partially Committed):** Questo stato viene raggiunto non appena l'ultima istruzione dello script viene eseguita, ma prima che le modifiche siano scritte in modo definitivo sul supporto fisico.
- **Impegnato (Committed):** La transazione si conclude con successo. Il DBMS ha memorizzato in modo permanente le modifiche nel registro di log persistente e conferma l'operazione all'utente.

- **Fallito (Failed):** Se il sistema rileva la violazione di un vincolo di integrità, o se si verifica un errore hardware o un blocco software (*deadlock*), la transazione interrompe la sua marcia regolare.
- **Abortito (Aborted):** È lo stato finale di una transazione fallita. Il DBMS avvia una procedura automatica di ripristino per annullare ogni minima alterazione provocata, riportando la base di dati allo stato iniziale coerente.

1.2 Le Proprietà ACID: I Quattro Pilastri dell’Affidabilità

Per garantire che la base di dati rimanga integra in qualunque scenario operativo, il modulo transazionale deve imporre il rispetto di quattro proprietà fondamentali, note universalmente con l’acronimo **ACID**.

1.2.1 Atomicità

L’atomicità esprime la regola ferrea del “tutto o niente”. Una transazione viene considerata dal DBMS come una singola particella indivisibile. Non è ammessa un’esecuzione parziale: se una transazione composta da dieci modifiche si interrompe alla nona a causa di un blackout, il sistema ha il dovere di cancellare anche le precedenti otto modifiche. Sotto il cofano, questo meccanismo viene gestito tramite un’operazione di *rollback*, che sfrutta i dati memorizzati nel registro di log per invertire le azioni compiute e ripristinare i valori originali dei record coinvolti.

1.2.2 Consistenza

La consistenza stabilisce che l’esecuzione di una transazione debba traghettare la base di dati da uno stato valido a un altro stato valido. Questo significa che al termine dell’operazione nessun vincolo di integrità (come le chiavi primarie, i vincoli di foreign key o le clausole **CHECK**) deve risultare violato. Sebbene durante la transazione i dati possano trovarsi in uno stato momentaneamente inconsistente (ad esempio, dopo aver sottratto il denaro dal primo conto ma prima di averlo accreditato sul secondo), l’applicazione e il DBMS cooperano affinché l’equilibrio strutturale venga ripristinato prima del salvataggio definitivo.

1.2.3 Isolamento

In un sistema reale, migliaia di utenti interrogano e modificano lo stesso database simultaneamente. L’isolamento garantisce che l’esecuzione di ciascuna transazione avvenga come se fosse l’unica attiva nel sistema, scudandola dalle interferenze causate dalle operazioni concorrenti. Il DBMS deve orchestrare i comandi intrecciati in modo tale che il risultato finale coincida esattamente con quello che si otterrebbe eseguendo le stesse transazioni una dopo l’altra, in modo strettamente sequenziale. I diversi livelli di isolamento gestiscono l’equilibrio tra il parallelismo delle prestazioni e la protezione dalle anomalie.

1.2.4 Persistenza (Durabilità)

La durabilità assicura che, una volta ricevuto il messaggio di **COMMIT** con successo, gli effetti della transazione diventino permanenti nel tempo e non possano più andare perduti. Il DBMS garantisce che tali aggiornamenti sopravvivano a qualsiasi tipo di guasto successivo, come crash improvvisi del sistema operativo, rotture hardware o interruzioni di corrente elettrica, facendo affidamento su strutture di memorizzazione stabili e non volatili.

1.3 L'Architettura Interna: Buffer Manager, Log e Protocollo WAL

Per comprendere come il DBMS possa garantire la persistenza senza affossare le prestazioni hardware del server, è necessario analizzare il funzionamento congiunto del **Transaction Manager** (che supervisiona il ciclo di vita delle transazioni), del **Recovery Manager** (deputato al ripristino in caso di crash) e del **Buffer Manager**.

Il Buffer Manager gestisce una porzione della memoria RAM centrale organizzata in blocchi, detti pagine di buffer. Quando una transazione richiede la lettura o la scrittura di un record, il DBMS non accede direttamente al disco fisso; carica invece l'intera pagina che contiene quel record nella RAM. Tutte le successive modifiche introdotte dai comandi **UPDATE** o **INSERT** avvengono inizialmente all'interno di questa memoria volatile. Le pagine modificate in RAM vengono marcate come "sporche" (*dirty pages*) e non vengono riversate immediatamente sul disco, poiché l'I/O meccanico o magnetico rallenterebbe drasticamente il sistema.

Questo approccio prestazionale introduce un rischio strutturale evidente: se il server si spegnesse improvvisamente, tutte le dirty pages andrebbero perdute, violando la proprietà di Durabilità per le transazioni già confermate. Per scongiurare questo pericolo in modo efficiente, i DBMS adottano il protocollo di sincronizzazione **Write-Ahead Logging (WAL)**, governato da due regole matematiche precise:

1. **La regola del WAL propriamente detta:** Il sistema non può riversare una pagina di dati modificata (dirty page) dalla RAM al disco se prima non ha provveduto a scrivere sul disco rigido la porzione del registro di log (*log record*) che descrive dettagliatamente quella specifica modifica.
2. **La regola del Commit:** Una transazione non può essere considerata nello stato di **Committed** finché tutti i record di log associati alle sue operazioni non sono stati fisicamente scritti e messi al sicuro sul supporto di memorizzazione stabile del log.

Poiché il file di log è una struttura dati puramente sequenziale (*append-only*), la sua scrittura su disco richiede uno spostamento minimo delle testine o dei blocchi di memoria, risultando infinitamente più rapida rispetto al salvataggio sparso delle pagine di dati reali.

In caso di crash improvviso del server, al momento del riavvio entra in azione il Recovery Manager, il quale effettua una scansione sequenziale del registro di log per ricostruire lo stato coerente del database attraverso due fasi distinte:

- **Fase di REDO (Rifare):** Il sistema individua nel log tutte le transazioni che avevano completato con successo il loro ciclo prima del guasto (ovvero quelle che presentano sia il record di **START** che quello di **COMMIT**), ma le cui pagine di dati si trovavano ancora nella RAM volatile e non erano ancora state scritte sul disco dal Buffer Manager. Il Recovery Manager riapplica fedelmente queste modifiche direttamente sul disco, blindando la Durabilità.
- **Fase di UNDO (Disfare):** Il sistema individua tutte le transazioni che si trovavano ancora in stato attivo o parzialmente impegnato al momento del crash (che presentano il record di **START** ma non quello di **COMMIT** o **ABORT**). Poiché queste transazioni sono state interrotte a metà, il Recovery Manager legge il log a ritroso e annulla ogni loro effetto sul disco, garantendo la proprietà di Atomicità.

2 Strutture Fisiche e Strutture di Accesso ai Dati

Quando ci allontaniamo dalla sintassi logica dell'SQL e scendiamo al livello dell'hardware, il database cessa di essere un astratto insieme di tabelle e si trasforma in una complessa architettura di file allocati su disco. Il problema ingegneristico centrale di qualsiasi DBMS è gestire il cosiddetto "collo di bottiglia dell'I/O" (Input/Output): leggere o scrivere un dato sulla memoria di massa è un'operazione meccanica o elettronica immensamente più lenta rispetto al processamento dei dati nella memoria RAM della CPU. Pertanto, l'intero scopo delle strutture fisiche è organizzare i byte in modo tale da minimizzare al massimo il numero di trasferimenti (pagine o blocchi) tra il disco e il Buffer Manager.

2.1 L'Organizzazione Primaria: Heap, Sequenziale e Hashing

L'organizzazione primaria definisce la disposizione materiale dei record (le tuple) all'interno dei blocchi del file al momento del loro inserimento. È la "casa madre" dei dati.

La struttura più elementare e utilizzata di default è l'**Organizzazione Heap** (a mucchio). In questo modello, il DBMS non impone alcun ordine logico o gerarchico. Quando una nuova tupla deve essere salvata, il sistema cerca banalmente il primo blocco del file che presenti sufficiente spazio libero e vi accoda il record. Sotto il cofano, questo rende le operazioni di inserimento fulminee, poiché non è necessario spostare o riordinare i dati preesistenti. Tuttavia, il prezzo si paga pesantemente in fase di interrogazione: per trovare una riga

specifica senza aiuti esterni, il motore SQL è costretto a spazzolare l'intero file dall'inizio alla fine, blocco per blocco. Questa operazione "forza bruta" prende il nome di **Scansione Sequenziale** (*Sequential Scan*) ed è il nemico numero uno delle prestazioni su tabelle popolate.

Un approccio diametralmente opposto è l'**Organizzazione Hash**. Qui, il DBMS rinuncia all'accodamento sequenziale e applica una funzione matematica (la funzione di hash) al valore della chiave del record. Questa funzione converte la chiave in un indirizzo fisico univoco, "calcolando" istantaneamente il blocco esatto in cui la tupla deve risiedere. L'hashing garantisce tempi di accesso costanti e pressoché istantanei per le ricerche di corrispondenza esatta (es. `WHERE matricola = 12345`). Il suo grave limite, tuttavia, è la totale cecità rispetto all'ordinamento logico: poiché i dati vengono dispersi matematicamente sul disco, l'hashing risulta del tutto inutile se l'utente richiede una ricerca per intervalli (es. `WHERE età BETWEEN 20 AND 30`), costringendo il sistema a scansionare tutto.

L'**Organizzazione Sequenziale**, invece, impone che i record vengano salvati fisicamente adiacenti e ordinati in base al valore di un campo chiave. Ottima per le letture ordinate, risulta però insostenibile nelle scritture: inserire una tupla nel mezzo del file richiederebbe lo slittamento fisico di tutti i record successivi, paralizzando il sistema.

2.2 Le Strutture Secondarie: Gli Indici e l'Albero B+-Tree

Per ottenere ricerche veloci senza paralizzare le scritture, i database relazionali si affidano alle strutture di accesso secondarie, universalmente note come **Indici**. Un indice agisce esattamente come l'indice analitico in fondo a un libro enciclopedico: non altera la disposizione del testo nelle pagine (la tabella Heap), ma crea una via di accesso rapida e separata, associando il termine cercato al numero della pagina fisica che lo contiene.

Il capolavoro ingegneristico e standard industriale per l'implementazione degli indici è l'**Albero B+-Tree** (B-Tree bilanciato). A differenza di un normale albero di ricerca binario studiato in programmazione, il B+-Tree è una struttura "tozza e larga", pensata appositamente per annidare migliaia di chiavi in pochissimi livelli logici, riducendo drasticamente i salti sul disco.

L'albero si sviluppa in tre strati:

- **Nodo Radice:** Il punto di ingresso della ricerca, caricato solitamente in modo permanente nella RAM.
- **Nodi Intermedi:** Contengono esclusivamente chiavi di instradamento (veri e propri "cartelli stradali" che indicano se proseguire la discesa a destra o a sinistra).
- **Nodi Foglia:** Si trovano tutti esattamente allo stesso livello di profondità (da qui il termine "bilanciato"). Contengono i valori reali indicizzati affi-

ancati dai *Record ID* (i puntatori hardware all'esatto blocco e riga della tabella principale su disco).

Quando esegui una query mirata (*Index Scan*), il DBMS parte dalla radice e scende lungo l'albero scartando intere porzioni del database a ogni bivio, con una logica di ricerca logaritmica. Che la tabella abbia mille righe o dieci milioni, il motore arriverà al dato con appena 3 o 4 letture sul disco.

Il vero colpo di genio del B+-Tree, rispetto agli alberi classici, risiede proprio nelle foglie: esse sono **concatenate tra loro sequenzialmente** (come una lista doppiamente concatenata). Una volta che la discesa nell'albero ha individuato il primo valore utile per una ricerca di range (ad esempio **prezzo** > 50), il sistema non deve ricominciare dall'alto per trovare i valori successivi; gli basta scorrere orizzontalmente la catena delle foglie recuperando agevolmente il 51, il 52 e così via. Questo rende il B+-Tree l'arma definitiva per risolvere filtri di disuguaglianza, le clausole **BETWEEN** e gli ordinamenti richiesti con **ORDER BY**.

2.3 L'Equilibrio tra Scrittura e Lettura: I Costi di Manutenzione

Sotto il cofano, l'adozione di un indice non è una panacea priva di effetti collaterali. Poiché l'albero è una copia speculare e rigorosamente ordinata dei dati, ogni volta che un'applicazione esegue un comando **INSERT**, **DELETE** o un **UPDATE** sul campo indicizzato, il DBMS è costretto a svolgere un doppio lavoro. Non solo deve modificare il record nella tabella Heap, ma deve anche aggiornare la struttura dell'albero per mantenerla matematicamente bilanciata, affrontando operazioni complesse come lo spaccamento dei nodi (*node splitting*) quando un blocco foglia si riempie.

L'uso indiscriminato degli indici accelera le letture in modo esaltante, ma zavorra inesorabilmente le prestazioni in scrittura. La vera abilità del progettista sta nell'osservare i piani di esecuzione e calare l'indice chirurgicamente solo sugli attributi strettamente necessari per l'incrocio dei dati (**JOIN**) o per le condizioni di filtro (**WHERE**) più frequenti.

3 Transazioni Concorrenti e Anomalie

Quando un DBMS gestisce carichi di lavoro elevati (centinaia o migliaia di transazioni al secondo), non può permettersi il lusso di eseguire le transazioni una dopo l'altra in sequenza stretta. Se lo facesse, mentre la CPU attende il disco per leggere un dato della Transazione A, tutte le altre transazioni rimarrebbero in coda, paralizzando il sistema. L'unica soluzione per ottenere alte prestazioni è l'**esecuzione concorrente**: il gestore della concorrenza (che agisce come un vigile urbano) intreccia le operazioni di lettura e scrittura delle varie transazioni in un flusso unico chiamato **schedule**.

Questo intreccio, tuttavia, è pericoloso. Se non controllato rigidamente, l'accesso simultaneo agli stessi dati provoca delle anomalie gravi che distruggono la coerenza del database.

3.1 Le Anomalie di Concorrenza

Per comprendere le anomalie, dobbiamo immaginare il database come una memoria condivisa. Utilizzeremo una notazione standard dove $r_i(x)$ e $w_i(x)$ indicano rispettivamente la lettura (*read*) e la scrittura (*write*) eseguite dalla transazione i sulla risorsa x . Le anomalie principali sono quattro:

- **Perdita di aggiornamento (Lost Update):** Si verifica quando due transazioni leggono lo stesso dato, lo modificano in memoria centrale e poi lo sovrascrivono l'una sull'altra in tempi diversi. L'effetto della prima transazione viene letteralmente spazzato via e perso nel nulla. Ad esempio, se T_1 e T_2 vogliono entrambe aggiungere 1 alla risorsa x (che vale 2): T_1 legge 2, T_2 legge 2, T_1 scrive 3, T_2 scrive 3. Il valore finale è 3 anziché 4.
- **Lettura inconsistente (Unrepeatable Read):** Avviene quando una transazione, durante il suo ciclo di vita, legge due volte la stessa risorsa ma ottiene due risultati diversi perché, nel frattempo, un'altra transazione ha modificato e consolidato quel dato.
- **Lettura sporca (Dirty Read):** È il problema più rischioso. Si presenta quando la transazione T_1 legge un dato che è stato appena modificato dalla transazione T_2 , ma T_2 **non ha ancora fatto il commit**. Se T_2 dovesse fallire e fare *abort*, T_1 si ritroverebbe ad aver basato i propri calcoli su un dato "fantasma" che di fatto non è mai esistito nel database.
- **Aggiornamento/Inserimento Fantasma (Phantom Insert):** È un'anomalia subdola legata agli insiemi di dati e non ai singoli record. Si verifica quando una transazione esegue una query statistica (es. "calcola la media dei voti") e, prima di concludere, ripete la stessa query ottenendo un risultato diverso perché, nel frattempo, un'altra transazione ha inserito una nuova riga che soddisfa i criteri della ricerca (un nuovo studente). La riga in più agisce come un fantasma che altera le medie.

Per evitare tutto questo, il DBMS deve garantire che lo schedule concorrente sia **serializzabile**, ovvero che produca esattamente lo stesso effetto di un'esecuzione puramente seriale (una transazione alla volta), pur mantenendo l'intreccio temporale a livello hardware.

3.2 La Teoria: Conflict-Equivalenza e Grafo dei Conflitti

Come fa il DBMS a capire matematicamente se uno schedule è sicuro? Si affida alla teoria della **Conflict-Equivalenza** (*CSR*).

Due operazioni appartenenti a due transazioni diverse si dicono **in conflitto** se accedono entrambe alla stessa risorsa x , e almeno una delle due è una scrittura. L'ordine in cui avvengono le operazioni in conflitto è vitale. Uno schedule è conflict-serializzabile se possiamo trasformarlo in uno schedule seriale scambiando solo le operazioni che *non* sono in conflitto tra loro.

Per verificare questa proprietà, si costruisce un **Grafo dei Conflitti** (o Grafo di Precedenza):

1. I nodi del grafo sono le transazioni attive (es. T_1, T_2). 2. Si disegna una freccia (arco) dal nodo T_1 al nodo T_2 se la transazione T_1 esegue un'operazione su una risorsa che va **in conflitto** con un'operazione successiva eseguita da T_2 sulla stessa risorsa.

Il teorema è semplice e potente: **se il grafo contiene un ciclo (un giro chiuso, come $T_1 \rightarrow T_2 \rightarrow T_1$), lo schedule è matematicamente invalido e non serializzabile**. Significa che T_1 dovrebbe avvenire prima di T_2 per una risorsa, ma T_2 dovrebbe avvenire prima di T_1 per un'altra, creando un paradosso temporale.

3.3 La Pratica: Protocollo Two-Phase Locking (2PL)

Siccome il DBMS non può prevedere il futuro e costruire il grafo prima che le transazioni finiscano, nei sistemi reali si usa un meccanismo di controllo preventivo basato sui **Lock** (i "lucchetti").

Prima di leggere un dato, una transazione deve chiedere un *r_lock* (lock condiviso: più transazioni possono leggere insieme). Prima di scriverlo, deve chiedere un *w_lock* (lock esclusivo: nessun altro può né leggere né scrivere). Se la risorsa è già bloccata in modo incompatibile, la transazione richiedente viene messa in attesa in una coda.

Il protocollo standard che garantisce la serializzabilità "al volo" è il **Two-Phase Locking (2PL)** (Locking a due fasi). Esso impone una regola geniale quanto rigida: **una transazione non può richiedere nessun nuovo lock dopo averne rilasciato almeno uno**. La vita della transazione si divide quindi in due fasi:

1. **Fase di Crescita:** La transazione acquisisce tutti i lock che le servono per lavorare. Non può rilasciare nulla. 2. **Fase di Decrescita:** Raggiunto il picco, inizia a rilasciare i lock. Da questo momento in poi, le è severamente vietato chiederne di nuovi.

Tuttavia, il 2PL semplice non protegge dalla lettura sporca (se una transazione rilascia un lock esclusivo in discesa e poi fallisce, un'altra potrebbe aver letto quel dato errato). Pertanto, i DBMS commerciali implementano lo **Strict 2PL** (2PL Stretto): tutti i lock esclusivi (*w_lock*) vengono trattenuti e rilasciati in blocco **solo e soltanto al momento del COMMIT (o del ROLLBACK)**. Questo blindi i dati finché non si ha la certezza matematica che la modifica è confermata e permanente.

3.4 Il Problema del Deadlock (Blocco Critico)

L'uso intenso dei lock introduce però una patologia chiamata **Deadlock** (Blocco Critico o stallo).

Si verifica quando due transazioni si bloccano a vicenda in un circolo vizioso. Immagina che T_1 abbia acquisito il lock esclusivo sulla risorsa A e abbia bisogno della risorsa B per finire. Nel frattempo, T_2 ha acquisito il lock esclusivo su B

e ha bisogno di A per finire. Nessuna delle due può andare avanti, ed entrambe aspetteranno all'infinito che l'altra ceda il passo.

I database gestiscono i deadlock in due modi principali:

* **Timeout:** Se una transazione aspetta un lock da troppo tempo, il DBMS la "spara" (la abortisce) facendole fare rollback. * **Rilevamento (Detection):** Il sistema controlla periodicamente la tabella interna dei lock cercando cicli di dipendenza. Se trova un deadlock, sceglie una transazione "vittima" e la abortisce per sbloccare l'altra, per poi riavviarla in un secondo momento.

4 Ottimizzazione delle Interrogazioni e Calcolo dei Costi Fisici

Nel panorama dei sistemi di gestione di basi di dati relazionali, il linguaggio SQL si distingue come un paradigma puramente dichiarativo. Quando scriviamo una query, specifichiamo dettagliatamente che tipo di dati vogliamo ottenere, ma lasciamo il sistema completamente libero di decidere la procedura ottimale per recuperare fisicamente tali dati dalla memoria secondaria. Il modulo interno incaricato di compiere questa delicata scelta strategica è l'ottimizzatore delle interrogazioni (*Query Optimizer*). Sotto il cofano del database, l'ottimizzatore analizza l'espressione dichiarativa, la scompone e la traduce in un piano di esecuzione fisico, strutturato come un albero di operazioni elementari destinate al gestore dei metodi di accesso.

4.1 Il Processo di Compilazione della Query

La trasformazione di una stringa di testo SQL in un codice eseguibile ottimizzato segue una vera e propria catena di montaggio software, articolata in quattro fasi sequenziali stabili:

1. **Analisi Lessicale e Sintattica:** Il DBMS verifica innanzitutto che la query rispetti la grammatica del linguaggio SQL e controlla nel dizionario dei dati che i nomi delle tabelle e degli attributi inseriti esistano realmente, convalidando i relativi permessi di accesso dell'utente.
2. **Ottimizzazione Algebrica:** La query viene tradotta in un albero logico basato sulle regole dell'algebra relazionale. In questa fase, indipendente dal contenuto effettivo del database, l'ottimizzatore applica euristiche matematiche universali. La regola d'oro consiste nell'*anticipo delle selezioni* e nell'*anticipo delle proiezioni*: spingendo i filtri (clausola **WHERE**) verso il basso dell'albero, il sistema riduce la cardinalità delle tabelle intermedie prima di eseguire le costose operazioni di combinazione (**JOIN**), abbattendo l'occupazione di memoria RAM nel buffer.
3. **Ottimizzazione Dipendente dai Metodi di Accesso:** Questo è il cuore computazionale del modulo. L'ottimizzatore esamina l'albero algebrico logico e genera molteplici varianti fisiche alternative, valutando per

ogni nodo quale algoritmo hardware implementare, ad esempio se per un filtro convenga una scansione lineare o un accesso tramite albero B+-Tree.

4. **Generazione del Codice e Scelta del Piano:** Sfruttando le formule di costo e i profili statistici delle tabelle aggiornati periodicamente dal DBMS, il sistema stima il numero di accessi alla memoria secondaria per ogni combinazione possibile, seleziona l'albero con il costo stimato minimo e lo invia al motore di esecuzione.

4.2 I Costi delle Operazioni di Accesso Elementari

Per quantificare matematicamente la convenienza di un piano rispetto a un altro, il DBMS utilizza come unità di misura fondamentale il numero di blocchi (o pagine) letti o scritti su disco, poiché il collo di bottiglia hardware risiede nell'I/O della memoria secondaria. Nei calcoli prestazionali, la dimensione di una relazione viene indicata con $NP(R)$ (Numero di Pagine) e il numero di righe totali con $NR(R)$ (Numero di Righe o Cardinalità).

4.2.1 Scansione Sequenziale (Sequential Scan)

Rappresenta l'operazione brute-force per eccellenza. Se una tabella non possiede indici a supporto o se il filtro coinvolge una porzione massiva dei dati, il gestore degli accessi deve leggere linearmente ogni singolo blocco allocato dal file system.

$$\text{Costo}_{\text{SeqScan}}(R) = NP(R)$$

4.2.2 Ordinamento Fisico (Z-way Sort-Merge)

L'ordinamento è un'operazione critica sotto il cofano, richiesta non solo dalla clausola `ORDER BY`, ma anche per eliminare i duplicati (`SELECT DISTINCT`) o per raggruppare i record (`GROUP BY`). Poiché le tabelle sono solitamente troppo grandi per risiedere interamente nella memoria RAM centrale, il DBMS adotta l'algoritmo di Sort-Merge esterno.

L'algoritmo opera in due fasi distinte:

- **Fase di Sort Interno:** Il sistema carica nel buffer un numero di pagine alla volta pari ai blocchi di memoria disponibili (NB), le ordina tramite algoritmi rapidi in RAM (come QuickSort) e scrive queste sequenze parziali ordinate (chiamate *run*) in file temporanei su disco. Questa fase richiede di leggere e riscrivere l'intera relazione, generando un costo iniziale di $2 \cdot NP(R)$.
- **Fase di Merge:** Le *run* vengono fuse progressivamente a coppie (merge a due vie, $Z = 2$) sfruttando i buffer di input e output del modulo fino a ottenere un unico file ordinato. Ogni passo di fusione legge e scrive l'intero volume di dati.

Il numero totale di passi di fusione necessari è determinato logicamente dalla formula $\lceil \log_2(NP(R)) \rceil$. Di conseguenza, il costo hardware complessivo dell'operazione è espresso matematicamente come:

$$\text{Costo}_{\text{Sort}}(R) = 2 \cdot NP(R) \cdot (\lceil \log_2(NP(R)) \rceil + 1)$$

4.2.3 Accesso Diretto tramite Indice

Quando filtriamo i dati tramite una condizione di uguaglianza o disuguaglianza su un campo indicizzato, il costo cessa di essere lineare e si trasforma in una costante o in un valore logaritmico controllato:

- **Indice Hash:** Ottimizzato per condizioni di uguaglianza atomica (**WHERE** $A = v$). Il costo teorico per scendere sul bucket puntatori è fisso:

$$\text{Costo}_{\text{Hash}} \approx 1 \text{ accesso}$$

- **Indice B+-Tree:** Supporta sia l'uguaglianza che le query per intervalli (**BETWEEN**). Il costo di discesa dipende esclusivamente dall'altezza dell'albero gerarchico, condizionata dal fattore di riempimento medio e dal numero di valori distinti dell'attributo, indicato con $VAL(A, R)$:

$$\text{ProfonditàIndice} \leq 1 + \log_{\lceil \frac{n}{2} \rceil} \left(\frac{CARD(R)}{VAL(A, R)} \right)$$

4.3 Gli Algoritmi Fisici di Join e le loro Formule di Costo

L'operazione di JOIN rappresenta il nodo più oneroso e computazionalmente critico nell'ottimizzazione fisica dei database relazionali. Date due relazioni in input, l'ottimizzatore assegna formalmente il ruolo di **Relazione Esterna** (R , posizionata nel ciclo più esterno) e **Relazione Interna** (S , scandita nel ciclo interno).

4.3.1 Nested Loop Join Standard

È l'algoritmo universale di base, applicabile a qualunque tipologia di predicato. Nella sua implementazione elementare (con una sola pagina di buffer allocata per tabella), il sistema legge interamente la tabella esterna R una volta sola. Successivamente, per ciascuna singola riga di R , è costretto a compiere una scansione sequenziale brute-force completa della tabella interna S sul disco.

$$\text{Costo}_{\text{NestedLoop}} = NP(R) + NR(R) \cdot NP(S)$$

Per minimizzare questo costo asimmetrico, l'ottimizzatore impone come relazione esterna R la tabella che presenta il minor numero assoluto di *righe* (NR).

4.3.2 Block Nested Loop Join

Per evitare di rileggere la tabella interna per ogni singola tupla, questa variante ottimizzata sfrutta la memoria del buffer leggendo i dati a blocchi di pagine. Il sistema carica una pagina intera della relazione esterna R nel buffer e, mantenendola fissa in RAM, esegue la scansione della tabella interna S confrontando le tuple riga per riga in memoria centrale.

$$\text{Costo}_{\text{BlockNestedLoop}} = NP(R) + NP(R) \cdot NP(S)$$

In questa configurazione strutturale, il costo è guidato dal numero di *pagine* (NP). Di conseguenza, la relazione esterna ideale deve essere quella con il minor numero di pagine complessive.

4.3.3 Nested Loop Join con Indice (Index Nested Loop)

Se la relazione interna S possiede un indice B+-Tree sul campo coinvolto nella condizione di join, il DBMS disattiva la scansione lineare di S . Per ogni riga della tabella esterna R , il sistema interroga l'albero della tabella interna estraendo chirurgicamente solo i blocchi in cui la chiave coincide, calcolati tramite la selettività statistica dell'attributo $\frac{NR(S)}{VAL(A,S)}$.

$$\text{Costo}_{\text{IndexNestedLoop}} = NP(R) + NR(R) \cdot \left(\text{ProfonditàIndice} + \frac{NR(S)}{VAL(A,S)} \right)$$

4.3.4 Merge Scan Join

Questo algoritmo fulmineo si applica esclusivamente in presenza di equi-join in cui entrambi gli input risultano già ordinati fisicamente (o tramite struttura primaria sequenziale ordinata o grazie alla presenza di un indice) sui campi di collegamento. Sfruttando la coincidenza dell'ordinamento, il DBMS fa scorrere due cursori paralleli sulle tabelle in un unico passo sincronizzato, leggendo ogni blocco una sola volta in assoluto:

$$\text{Costo}_{\text{MergeScan}} = NP(R) + NP(S)$$

5 MongoDB: Architettura Distribuita e NoSQL

A differenza dei tradizionali sistemi relazionali, MongoDB è un database NoSQL orientato ai documenti. Questo significa che le informazioni non sono rigidamente ingabbiate in tabelle bidimensionali fatte di righe e colonne, ma vengono salvate come documenti indipendenti (simili a file JSON). Sotto il cofano, l'architettura di MongoDB è stata ingegnerizzata attorno a due pilastri fondamentali per gestire enormi carichi di lavoro moderni: la replicazione, per garantire che i dati non vadano mai persi (affidabilità), e lo sharding, per distribuire il peso computazionale su più macchine (scalabilità orizzontale).

5.1 Replicazione e Alta Affidabilità (Replica Set)

Per evitare che il guasto di un singolo server provochi la paralisi del sistema o la perdita irreparabile dei dati, MongoDB non salva mai un'informazione in un unico luogo. Utilizza invece i **Replica Set**, ovvero dei cluster composti da più nodi (server) che mantengono costantemente una copia speculare dello stesso database.

All'interno di questo gruppo, l'architettura impone una rigida gerarchia. Esiste un solo **Nodo Primario**, che funge da punto di ingresso esclusivo per tutte le operazioni di scrittura inviate dalle applicazioni. Quando il primario riceve un comando di modifica, lo esegue e ne registra istantaneamente i dettagli in un registro speciale chiamato **oplog** (operations log). I **Nodi Secondari** si comportano come dei fedeli gregari: leggono continuamente l'oplog del nodo primario e riapplicano le stesse identiche operazioni sui propri dischi, mantenendosi sincronizzati. Essendo l'oplog strutturato per essere idempotente, anche se un nodo secondario dovesse applicare per errore due volte la stessa operazione, il risultato finale non verrebbe alterato.

Il sistema si monitora da solo tramite un meccanismo di *heartbeat*: i nodi si inviano reciprocamente dei piccoli segnali di vita ogni due secondi. Se il nodo primario subisce un guasto hardware o viene isolato dalla rete, i nodi secondari smettono di ricevere il suo battito cardiaco. Dopo un tempo di timeout (generalmente 10 secondi), il cluster si rende conto dell'emergenza e innesca automaticamente un algoritmo di elezione basato sul protocollo di consenso **Raft**. I nodi superstiti votano tra di loro e il nodo secondario che possiede i dati più aggiornati viene promosso al ruolo di nuovo primario. Per evitare situazioni di stallo durante le votazioni (il classico "pareggio"), le best practice impongono che un replica set sia sempre composto da un numero dispari di nodi votanti; se per motivi di budget si hanno solo due server dati, si inserisce un nodo *Arbitro* molto leggero, il cui unico scopo è esprimere un voto decisivo in caso di elezione, senza però salvare alcuna copia dei dati.

Per permettere agli sviluppatori di bilanciare la velocità con la sicurezza, MongoDB introduce il concetto di **Write Concern** (garanzia di scrittura). Quando un'applicazione inserisce un dato, può dire al database: "Considera l'operazione conclusa solo dopo che il dato è stato salvato sul primario" (massima velocità, ma rischio di perdita se il primario esplode un istante dopo), oppure può esigere un livello *majority*, dicendo: "Mettimi in attesa e dammi conferma solo dopo che la maggioranza assoluta dei nodi ha copiato fisicamente il dato sui propri dischi" (più lento, ma matematicamente a prova di bomba).

5.2 Sharding e Scalabilità Orizzontale

Mentre la replicazione protegge dai guasti, lo **Sharding** risolve il problema dello spazio e della potenza di calcolo. Quando un database diventa così titanico da non poter più risiedere fisicamente sul disco rigido di un singolo server, o quando le query al secondo saturano la CPU, MongoDB "fa a fette" l'intera collezione di dati, distribuendo i frammenti (chiamati *chunk*) su più server indipendenti

detti **Shard**.

Questa architettura distribuita è invisibile all'utente finale grazie a un componente geniale chiamato **mongos**. Il *mongos* agisce da router intelligente e da vigile urbano: l'applicazione client non parla mai direttamente con i server che contengono i dati, ma invia la query al mongos. Quest'ultimo consulta istantaneamente un **Config Server** (il cervello del sistema, che contiene la mappa esatta di dove si trova ogni frammento di dato), capisce a quale shard inoltrare la richiesta, raccoglie i risultati parziali, li unisce e li restituisce all'utente in un unico pacchetto pulito.

Il modo in cui MongoDB decide come affettare e distribuire i documenti è governato dalla **Shard Key**, una chiave scelta dal progettista. Sotto il cofano, esistono due strategie matematiche opposte per distribuire i dati:

- **Ranged Sharding (Distribuzione a intervalli):** I dati vengono tagliati in base a range continui della chiave (es. "Cognomi da A a M sul Server 1, da N a Z sul Server 2"). È un approccio straordinariamente efficiente se l'applicazione fa molte interrogazioni per intervalli. Tuttavia, se si sceglie una chiave che cresce in modo sequenziale (come un ID numerico o una data di registrazione), tutto il nuovo traffico in entrata andrà a martellare esclusivamente l'ultimo server che gestisce il range più alto, lasciando gli altri inattivi.
- **Hashed Sharding (Distribuzione calcolata):** MongoDB prende la chiave, la passa attraverso una funzione crittografica di hash e la trasforma in una stringa alfanumerica pseudo-casuale, usando quel risultato per assegnare il server. Il grandissimo vantaggio è che dati inseriti in sequenza vengono "spruzzati" uniformemente su tutti i server della server farm, garantendo un bilanciamento perfetto del carico hardware. Il rovescio della medaglia è che le query per intervallo diventano disastrose: per trovare tutti gli utenti registrati in un certo mese, il *mongos* è costretto a interrogare tutti i server contemporaneamente, poiché i dati sono stati matematicamente dispersi.

Per mantenere l'equilibrio nel tempo, un processo silenzioso in background chiamato **Balancer** vigila sul sistema. Se si accorge che uno shard si sta riempiendo di chunk molto più velocemente degli altri, interviene spostando fisicamente i frammenti di dati da un server all'altro. Se però un chunk diventa troppo grande e non divisibile perché tantissimi documenti possiedono l'identico valore della chiave di sharding, si crea un *Jumbo Chunk* che il balancer non riesce a spostare, richiedendo l'intervento manuale di un amministratore.

5.3 Il Modello Transazionale e le Proprietà ACID

Adattare le rigide proprietà ACID (Atomicità, Consistenza, Isolamento, Persistenza) tipiche dei database relazionali in un ambiente intrinsecamente frammentato e distribuito come MongoDB richiede compromessi architetturali specifici.

La differenza più radicale riguarda l'**Atomicità**. In MongoDB, l'atomicità è garantita strutturalmente a livello di *singolo documento*. Se un documento JSON contiene al suo interno un'intera gerarchia di sotto-documenti e array (come una fattura che include tutte le sue righe di prodotto), qualsiasi aggiornamento che colpisce quel documento sarà sempre e comunque un'operazione del tipo "tutto o niente". Questo riduce drasticamente la necessità di aprire transazioni complesse su più tabelle, giustificando il limite ingegneristico di 16 Megabyte di peso massimo imposto per ogni singolo documento.

Per quanto riguarda l'**Isolamento** e la **Consistenza**, il motore permette di calibrare il comportamento in base alle necessità della singola query tramite il *Read Concern*. Si può optare per la *Strong Consistency* (il database garantisce matematicamente di restituire il dato più fresco in assoluto, al prezzo di una latenza maggiore in lettura) o per la *Eventual Consistency* (il sistema risponde alla velocità della luce interrogando un nodo secondario, accettando il rischio calcolato di restituire un dato vecchio di qualche millisecondo che non è ancora stato sincronizzato).

Infine, per garantire la **Persistenza** (Durability), il motore di archiviazione salva i dati nella RAM volatile per massimizzare le prestazioni, ma si protegge dai guasti hardware implementando un meccanismo di Write-Ahead Logging (WAL): i registri logici di tutte le operazioni vengono forzatamente scritti sul disco fisico ogni 100 millisecondi, permettendo al gestore dell'affidabilità di ricostruire perfettamente lo stato coerente del database in caso di un brutale blackout elettrico.